

# GrantGenie

Josh Norlin  
University of Idaho  
Moscow, USA  
norl6435@vandals.uidaho.edu

Carson Gustavel  
University of Idaho  
Moscow, USA  
gust1185@vandals.uidaho.edu

## Abstract

This document focuses on the creation and development of a grant management system (GMS) called GrantGenie. This project is part of the curriculum for the class CS 3600 at the University of Idaho. The goal of the project is to allow principle investigators (PIs) to manage federally-awarded research grants in compliance with both federal and institutional rules.

GrantGenie will be developed with the front-end framework React.js. The backend will be developed with Supabase, a backend-as-a-service. GrantGenie will also use Supabase's hosted PostgreSQL as the database management system in order to store and process data from users.

There will be a large focus on key aspects such as regulation enforcements, user authentication, and transaction approval. GrantGenie is currently a fully functional platform. It includes features that allow users to enter, validate, and view grant transactions, as well as create and export grant budgets to aid with compliance and auditing. GrantGenie also includes many user management features, such as email registration and verification, and a grant invitation system that allows PI's to add other users to a grant.

## CCS Concepts

• **Information systems** → **Database design and models**; *Database web interfaces*; • **Software and its engineering** → **Middleware**; • **Human-centered computing** → *Web-based interaction*.

## Keywords

Grant Management Systems, Research Administration, Subaward Tracking, Database Schema Design, Access Control, Data Integrity, Web Middleware, RESTful APIs, Workflow Automation, Edge Computing

### ACM Reference Format:

Josh Norlin and Carson Gustavel. 2025. GrantGenie. In *The 40th ACM/SI-GAPP Symposium on Applied Computing (SAC '25)*, March 31–April 4, 2025, Catania, Italy. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3555776.3577625>

## 1 Introduction

The requirements for this project include that a user can create a grant and assign monetary limits to each budget category. The

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

SAC '25, Catania, Italy

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-9517-5/25/03

<https://doi.org/10.1145/3555776.3577625>

user can enter a request to spend from a specific budget line. This will be done through an LLM agent interface. The system will validate the request against federal and institutional rules in real time. Disallowed requests will be flagged with an explanation.

The project also requires that the application produce up-to-date budget status reports. These reports will show committed and remaining funds by category. GrantGenie must be able to generate exportable Excel spreadsheets for university accounting and auditing purposes. Additionally, the system must allow for subawards by supporting management for subrecipients, enforcing prime award and subaward-specific rules.

Lastly, the system will allow users to create password-protected accounts containing personal details such as phone number, and email. A PI can invite multiple users to work on a grant together and assign them roles. This supports the enforcement of prior-approval transaction rules.

## 2 ER Diagram

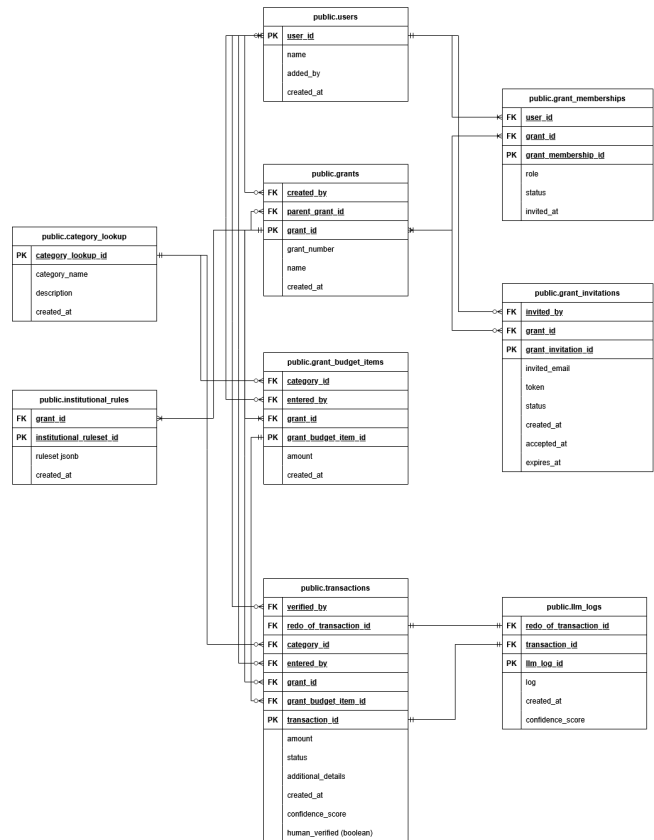


Figure 1: Entity Relationship Diagram

Our ER diagram (Figure 1) encompasses the logical process of how GrantGenie handles information. There are six entities: *Users*, *Grants*, *Restriction Lists*, *Categories*, *Grant Budget Items*, and *Transactions*.

- **Users:** Attributes include `user_id` (primary key), `first_name`, `last_name`, and `created_at`.
- **Grants:** Contains `grant_id` (primary key), a reference to `user_id`, `parent_grant_id`, and `created_at` attributes. The parent grant ID attribute is self-referential, allowing for sub-awards.
- **Grant Memberships (Helper Table):** Contains `grant_id` and `user_id` as foreign keys. The table is used as a search table to satisfy RLS policies when sensitive data is being dealt with.
- **Restriction Lists:** Attributes include `restriction_id` (primary key), a reference to `grant_id`, `restrictions`, and `created_at`.
- **Categories:** Attributes include `category_id` (primary key), and `category_name`.
- **Transactions:** Contains `transaction_id` (primary key), references to `grant_id` and `category_id`, `status`, `amount`, and `created_at` as attributes.
- **Grant Budget Items:** Contains `budget_id` (primary key), references to `grant_id` and `category_id`, `created_at`, and `amount` as attributes.
- **LLM Logs:** This table contains a foreign key reference to the transaction table's `transaction_id`. This contains contains LLM-generated logs for each entered transaction.
- **Grant Invitations:** This table allows for PIs to invite other users. The main attributes include token details like time-to-expiration and general timestamps, and foreign key references like `grant_id` and `invited_by`.

Relationships include:

- *Users & Grants:* Links users to grants (many-to-many). A grant must be associated with one user for it to exist. This relationship allows PIs to add multiple users to the same grant.
- *Grants & Restriction Lists:* Links grants to their institutional restrictions (one-to-one). This relationship supports institution-specific rule enforcement after entering a transaction.
- *Grants & Transactions:* Links grants to their entered transactions (one-to-many). This allows for exportable documents and budget limit calculations after totaling transaction amounts. The relationship also allows for PIs to see how much they've spent, which will feed directly into the grant management dashboard.
- *Transactions & Categories:* Links transactions to their associated categories (one-to-many). The Categories table is a static reference table. This relationship ensures transaction categorization, which allows for grant budgets to be split among set categories.
- *Grant Budget Items & Categories:* Links categories to grant budgets (one-to-many). This connection allows PIs and other staff to assign monetary limits to different categories when initializing the grant.

- *Grants & Grant Budget Items:* This relationship connects grants to their respective grant budgets (one-to-many). This enables PIs to see the monetary activity of the grant, combined with the Grants & Transactions relationship.

### 3 Schema Normalization

Our schema was designed to reduce data redundancies and maintain data integrity. Each table models a single entity (such as grants, users, transactions, etc.), and attributes are arranged to avoid update, insertion, and deletion anomalies. As a result, the schema naturally follows standard normalization principles up to Third Normal Form.

Each table has a defined primary key and contains only atomic values. For example, a grant's ID, name, and creation date are stored as individual attributes. We also avoided repeating groups by separating them into related tables, instead of storing multiple values, like grant categories, in a single field. The implementation of these database-design principles ensured First Normal Form, the baseline for all database schema.

Since all major entities use single-column primary keys (e.g., `grant_id`, `user_id`), partial dependencies cannot occur. Any tables with composite keys only store foreign keys and contain no non-key attributes, ensuring Second Normal Form.

Lastly, the schema also adheres to Third Normal Form. Non-key attributes depend solely on the table's primary key. For example, user details (name, email address, etc.) depend only on `user_id` and are stored in the Users table rather than being duplicated in every grant. Similarly, grant transaction information is isolated in the Transactions table, preventing transitive dependencies where grant records would otherwise rely on grant information indirectly.

### 4 Sequence Diagrams

A sequence diagram is a type of UML diagram that is used to show the interactions between components in a system over time.

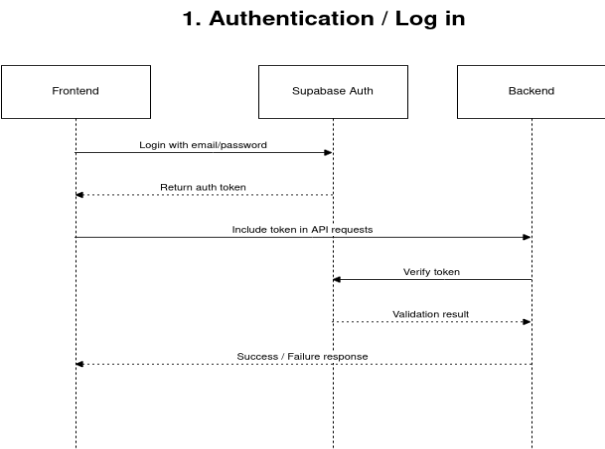
The following are a series of sequence diagrams that describe the main functions of Grant Genie (Figures 2-10). These are an important part of the design, showing how various components of Grant Genie will function and interact chronologically during real use cases. These functions include account management (login, user creation, etc), grant management functions (creation of grants and budget items), and transaction management functions (creation and validation). The former two are primarily for setup and the latter is the primary function of Grant Genie.

A full set of sequence diagrams are found at "GrantGenie/documentation/sequence\_diagrams/" within the source code.

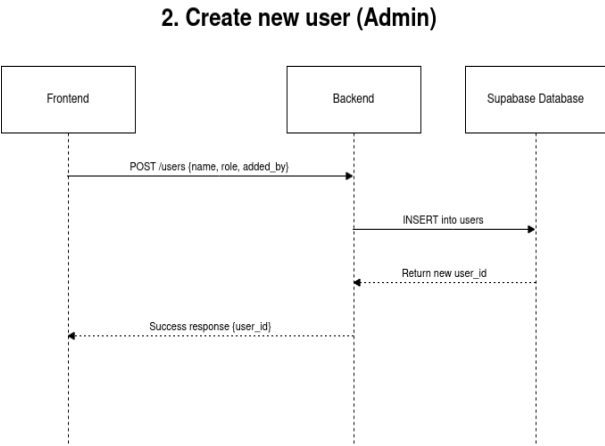
### 5 Tools

The main development tool for the application's front-end is React.js. React.js is a JavaScript web development library that allows for reusable, integrated components to create coherent interfaces and a seamless user experience.

Supabase will be used for the backend, business logic, and database. Supabase is a backend-as-a-service development platform that offers many ready-to-use, open-source tools for developers. Lastly, Supabase is accessed entirely through the front-end, with security measures put in place to ensure back-end safety. Supabase PostgreSQL offers a hosted database with easy-to-implement Row



(a) Authentication/Login Sequence



(b) Admin User Creation Sequence

Figure 2: A pair of sequence diagrams involved in account management

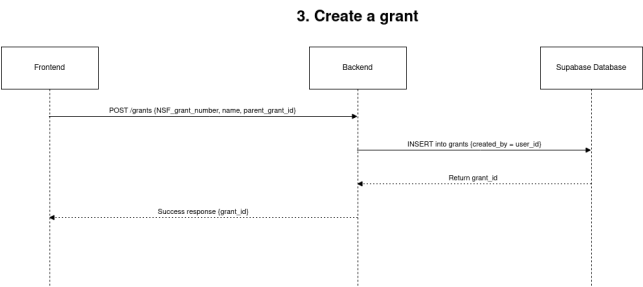


Figure 3: A diagram showing the sequence required to create a grant (note: a similar process is used in creating budget items as well).

Level Security policies and role management. Supabase’s database product is built using PostgREST, a web server that offers ready-to-use database APIs. This makes it perfect to store and retrieve user

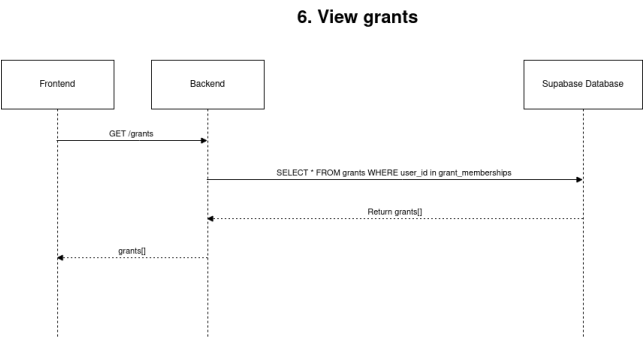


Figure 4: A diagram showing the sequence required to retrieve grant information (note: a similar process is used in viewing budget items and transaction data as well).

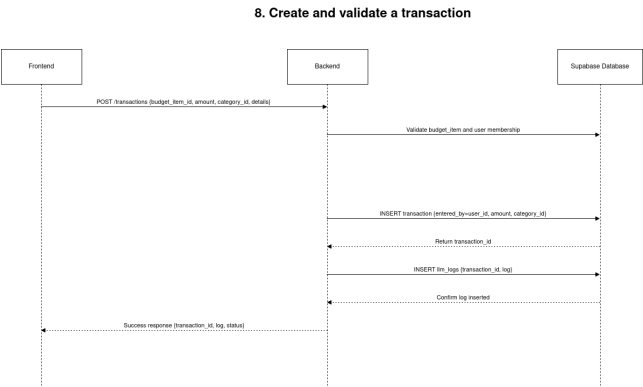


Figure 5: A diagram showing the sequence required to create and Validate a transaction (note: this is the primary function of GrantGenie).

and grant data. Supabase also has an auth feature that integrates with their PostgreSQL database. The project will use this feature to setup different roles for grants, allowing for chain-of-command grant transaction approval processes. Supabase Auth uses JSON web tokens (JWTs) to authenticate users, making the application fast and scalable.

Supabase not only covers database management, security, and hosting, but also covers backend functionality and business logic through Edge Functions. Edge Functions are Supabase’s server-side Typescript functions. These functions can handle HTTP requests and orchestrate calls to LLM APIs, allowing for PIs to enter grant transactions and view their statuses in real time.

Lastly, the project will use Google Gemini to turn grant project descriptions into LLM-friendly institutional restrictions that will be utilized at the submission of every transaction. Additionally, Google Gemini will also assist in reconciling grant transaction information with the federal and grant-specific institutional rules previously generated. This is the core tool that will allow PIs to comply with federal and institutional restrictions.

## 6 Backend Architecture and Business Logic Execution

GrantGenie uses Supabase's Edge Functions for business logic and relevant API calls. Edge Functions are server-side logic that runs on a locally-distributed Supabase Edge Node closest to the user's location, allowing for minimal latency. Edge Functions are typically used for Supabase APIs (Auth, Storage, Database, etc) and other third-party APIs.

### 6.1 Handling LLM-orchestrated Transactions

GrantGenie makes use of one edge function to handle the entire transaction process. When the user submits a transaction, the frontend calls and sends the edge function the following inputs:

- `user_id` (the person submitting the transaction),
- `grant_id`,
- `category_id`,
- `transaction_amount`,
- and the user-entered `transaction_description`

Once the edge function receives the input, a complex series of processes are executed to correctly log all relevant information and talk to the LLM. The edge function, named `verify-transaction`, does the following processes, which will be immediately explained further and architectural choices will be justified:

- (1) Validate membership
- (2) Gather all relevant context
- (3) Check transaction against budget constraints
- (4) Send and receive content to / from Google Gemini API
- (5) Handle necessary logging and returning

As mentioned above, the edge function verifies the transaction submitter is a part of the grant they are submitting for. This is done by searching the `grant_memberships` table with `user_id` and `grant_id`, and looking for a corresponding entry. This initial security check is done to obviously prevent users from inserting disallowed transactions.

The next step is to insert a transaction entry into the `transactions` table. The initial input is added into the corresponding fields, and a transaction status of 'pending' is set. Once this happens, the context fetching begins.

In order to maximize the actual accuracy of the decision-making LLM, the app tries to give it as much applicable context without significantly bloating the response time. The edge function fetches the **budget line item**, the **institutional ruleset**, **all available category information**, and **all recent transactions**. The first three inputs for the LLM are necessary and are relatively trivial. GrantGenie's extended capabilities can be seen in the fourth input, the recent transactions. The edge function fetches all transactions for a grant for one category made in the past 90 days. Some processing logic puts the transactions in an array, and sends it to the LLM for context. GrantGenie uses recent transactions as context to add an aspect of time. This significantly extends the possibilities for model-able university rules. With this in place, rules like "a PI cannot spend more than X% of the budget over the past 30 days" become possible.

Before the curated prompt and relevant context are sent to the LLM, a quick budgetary check must be performed. The edge function fetches the cumulative amount spent, adds the pending transaction amount to that, and compares the total to the budgeted amount. If the total is higher than the allotted amount, the transaction would put the grant over-budget, and is correctly rejected.

Once all checks are successfully completed, the context and a prompt is sent to Google Gemini. The prompt is quite long, but it essentially tells the LLM agent to be a grant investigator, and verify that the attached context / transaction follows all federal grant-related rules and institutional restrictions. The LLM agent is also informed it should use the user-entered description for justification, provide a confidence score for the overall transaction, and give feedback on additional necessary information to improve the confidence score.

Lastly, the edge function logs the LLM's output and the result of the transaction for auditing purposes.

### 6.2 Handling Grant Invitations

Once again, GrantGenie uses a series of edge functions to handle grant invitations. The first edge function, `create-invite`, generates a random token for each request, and makes an entry into the `invitations` table. Things like expiry date, and other details are automatically added on token creation. The use of tokens allows for safe, encrypted grant invitation.

The other edge function, `accept-invite`, handles the other end of the invitation system, accepting invites and managing the token life cycle. When the edge function is called, when an invited user clicks the "accept" button, it first checks whether the token has expired or not. If the token has not expired, the function continues on and inserts an entry into the `grant_memberships` table.

The synergy of these edge functions allow PIs to add other users to grants, which is a core functionality and is required in the syllabus.

## 7 Plan and Timeline

This project was started in the 2025 Fall semester at the University of Idaho. Development is split into several phases, with this document marking the end of Phase I. The project will continue throughout the course of the semester and ultimately conclude at the end of the semester with a complete product.

- **October 28-30:** By this deadline, the project should include a working prototype to simulate PIs (as well as other staff), grant initialization, transaction approval, and budgeting interfaces. The system should have a minimum front-end to integrate and simulate the core features. Additionally, the user management subsystem shall be implemented, allowing for users to sign up as a PI/grant admin, verify their account, and add users to a grant via email links. PIs shall be able to add and revoke grant users' permissions, creating an institutional-like management structure. Lastly, this information should be properly shown in a Phase II document and presentation.
- **November 18-20:** By this time, the project should have advanced to a fully-functional application with fully-developed

designs. The project should display all interfaces and properly enforce federal and institutional rules for all grant initializations and transactions. Again, the project and its functionality should be properly conveyed in a Phase III document and presentation.

- **December 9-11:** By this deadline, the final project should be released. The final touches for the product will be implemented. The product should be fully-functional, with adequate documentation. Also, a list of future improvements and known bugs may be created with plans to fix the bugs. All will be included in a Phase IV document and presentation.
- **Edits to Schedule:**
  - Phase II deadline was originally October 21-23, later rescheduled to October 28-30.

## 8 Subsystem Design

GrantGenie's frontend can be distinctly defined into three subsystems: Account/User Management, Grant Management, and Transaction Management. The following subsections further breakdown and give a clear picture of each corresponding subsystem.

## 9 Phase II Progress Report

This section will discuss the state of Grant Genie as of the extended Phase II deadline (November 2, 2025). Despite progress on the database and front-end, as well as a far more refined design, the current state of the project does not meet expectations.

Most of the work so far has been on refining the design of Grant Genie, and setting up the various components (Supabase PostgreSQL, Supabase Auth, React.js with TS, and Google Gemini) and preparing each for development and integration. The database tables have been setup in Supabase, and a basic frontend is working.

## 10 Phase III Progress Report

This section will discuss the state of Grant Genie as of the Phase III deadline (November 20, 2025). The database and backend continue to work as intended, though some changes have been made to the schema in order to accommodate practical considerations with the front-end and LLM. The backend now connected with Google Gemini and can query the LLM. The next step is to create UI to better facilitate the use of the new features: allowing users to properly submit, view, and approve transactions with the user. The LLM prompts also need to be tested and refined in order to ensure proper validation of transactions.

## 11 Phase IV Progress Report

This section will discuss the state of Grant Genie as of the Phase IV deadline (December 19, 2025). All features are currently operational; GrantGenie is now . Leading up to the demo, refining the UI was the main priority, but progress was also made on the invitation and personnel management features. During the demo, the LLM was unable to properly accept transactions due to overly restrictive institutional rules. Now, after tweaking the test set, transactions are now accepted and rejected at a more balanced rate. Though further testing would be required for an enterprise app, we are happy with the state of LLM's decision making.

Figure 6: The "Grant Creation" modal.

## 12 Project Limitations

While GrantGenie provides significant value to grant PIs, the project does have some limitations and heavily depends on some assumptions. Firstly, the grant management system heavily relies on the use and accuracy of current LLMs. Significant improvements in generative AI models have been made, but these machines are relatively unstable and highly variable, making it difficult to get consistent results given slight variations in input. Additionally, LLM models will inevitably "hallucinate" slightly in every answer, so some transaction evaluations may be incorrect, necessitating an user to review every transaction.

Some more limitations exist due to the initial decisions made regarding the project's architecture. GrantGenie requires PIs to upload their grants' budgets using a certain format (specific categories, CSV format, etc). If the budget was originally created in a different format, then adjusting the budget to meet GrantGenie's budget specifications may cause a loss of information (e.g. use of different category labels, different assigned meanings for categories, etc).

## 13 Frontend Design and Features

The front-end of GrantGenie provides many features such as account creation, login/authentication, and grant and budget creation, alongside the primary feature of transaction analysis using an LLM. All core, customer-facing features of the GrantGenie database are available to authorized parties using the frontend; there is no need for anyone to access supabase or other backend features besides the developers.

### 13.1 Grants Page

The "Grants" page allows users to create grants and budgets. The "Create a New Grant" interface allows authorized users to add grants to the database by submitting the grant's name and award number. The submitted grant, alongside any other grants associated with the current user will appear in a list below this interface. The grant submission modal allows the entering of grant data in 5 steps: "Grant Details", "Budget", "Institutional Rules", "Invite Members", and "Review and Finalize". The "Grant Details" allows the entering of the grant name and award number. "Budget" allows the user to optionally submit budget amounts for various categories. Budgets may also be entered or edited later in the process using the grant view modal after creation. The institutional rules step is essentially just a placeholder, but would allow for the entering of specific

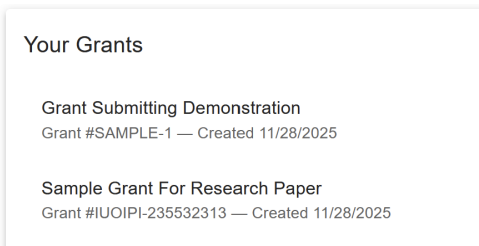


Figure 7: A sample of several submitted grants (Design as of Phase III)

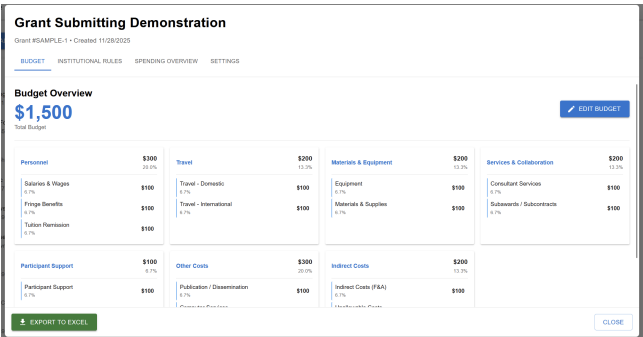


Figure 8: The budget viewing tab of the "Grant Details" modal

institutions or rules depending on the grant's specific restrictions. "Invite Members" would allow the user to invite other members to the grant using email verification. Finally, "Review and Finalize" allows the user to view all entered data and submit the grant. All data, except for the grant's title and review number, is subject to change at a later time by users with proper authorization using the "Grant View" modal.

Each grant from the list can be clicked on to reveal further details, submit a budget, or review the institutional rules assigned to the grant. Within the "Grant Details" module, users can view or enter budget line items in the budget entry tab. Budget line items are broken down by a limited set of categories in order to facilitate the LLM authentication. Other categories could be added in future versions, though this may complicate testing.

### 13.2 Transactions Page

The transaction page allows the user to submit and review transactions pertaining to their grants. The image includes a list of grants and transactions, a basic manual approval system, and a grant submission system.

The user can submit a transaction pertaining to a particular grant based on its category, amount, and a brief description of the transaction. On submission, these values are used alongside other previous transactions, the grant's budget, institutional rules, and other grant information to automatically evaluate the submitted transaction upon submission.

Upon completion of the Supabase edge function, the result is displayed to the user within the submission modal. The resulting data

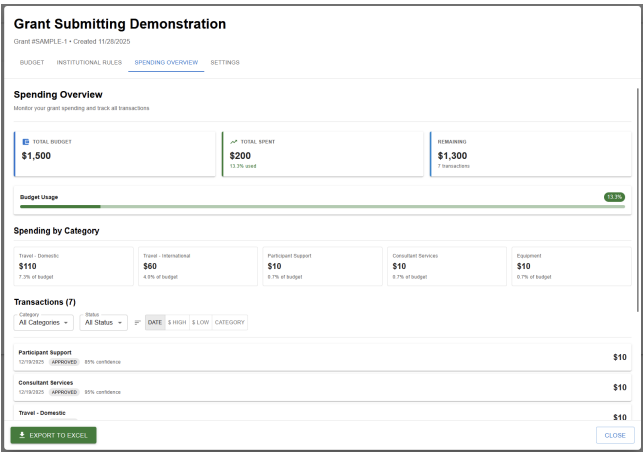


Figure 9: The spending overview tab of the "Grant Details" modal.

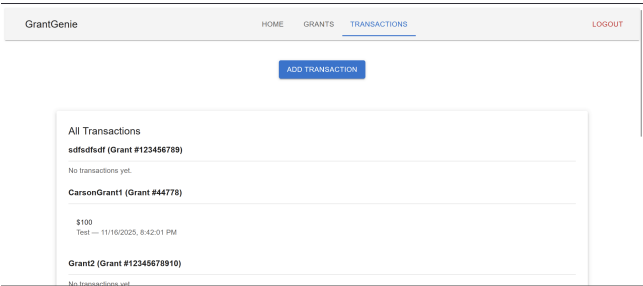


Figure 10: An image of the Transaction Page. (Design as of Phase II)

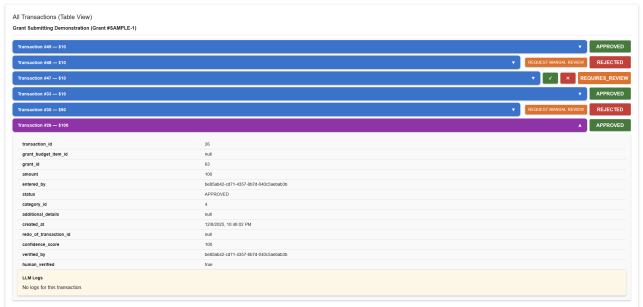
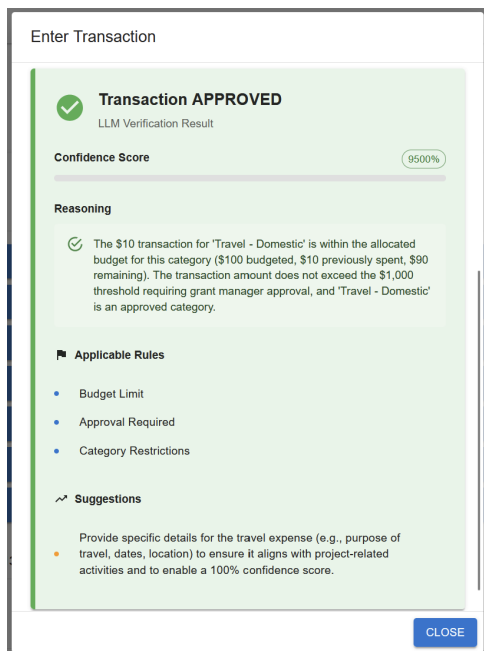


Figure 11: A screenshot of the transaction list. Transactions are shown with varying states of approval, review, and rejection.

includes the decision verdict, reasoning for the verdict, a confidence score, and other relevant data.

Transactions can be submitted for review using the "Enter Transaction" modal window. Once the user has completed selection of the grant and grant category, and entered the amount and description, the transaction can be submitted. Upon submission, a transaction



**Figure 12: Example of a transaction that was approved using the LLM review feature**

is sent to Google Gemini using a Supabase edge function, allowing for automatic approval and rejection.

Through the "All Transactions" table view, a user can view their grant transactions. Only the transactions associated with grants pertaining to the current user are shown. Each transaction has a drop-down which displays relevant information about the grant. Transactions are displayed differently depending on their status:

- (1) Accepted transactions are indicated with green. They have no special actions available; these transactions are good to go.
- (2) Rejected transactions are indicated with red. They can be resubmitted for manual review using a button, changing their status from REJECTED to REQUIRES\_REVIEW.
- (3) Transactions that require review are indicated with Orange and can be approved or rejected using a red "X" or green "✓" button by authorized users. Manual approval or rejection will not only change the status of the transaction, but also associate the verification with the user that approved it to allow better tracking of manual approval.
- (4) Pending Transactions are indicated with orange and have no special functionality. They will change to one of the other categories once LLM verification is complete.

### 13.3 Invitations Page

An "Invitations" page allows the user to accept or decline grant invitations. Invitations can be sent via the "Grants" page, and are handled through the emails associated with each account.

### 13.4 Exported Budget Spreadsheets

One feature of GrantGenie is the ability to export grant budget spreadsheets. These can be used for auditing, alternate storage of data, or as simply another way to view the data generated by GrantGenie.

An Exported spreadsheet contains four sheets:

- **Budget Summary:** a table including data columns for each used budget category: category name, amount budgeted, amount spent, amount remaining, and percent utilized.
- **Transactions:** a table including data columns for each transaction pertaining to the grant: transaction id, date, category, amount, status, the user who entered it, and other details.
- **Budget Items:** a table including data columns for each used budget category: category name, budgeted amount, and date entered.
- **LLM Logs:** a list of LLM logs pertaining to transactions from the given grant. Important columns include date, amount, status, and a full content of the LLM log (including reasoning, confidence level, suggestions, etc).

## 14 Future Research Directions

Although GrantGenie has achieved the core objectives of creating a functional grant management system with a normalized database schema, authenticated user workflows, and an intuitive interface, there are several areas for enhancement and expansion. These fall into functionality improvements, database-level improvements, performance optimizations, and organizational-level features.

- (1) **Document Management and File Storage Integration:** Currently, document storage is not supported. Future versions could include:

- File uploads (PDFs, Word docs) for applications, proof of eligibility, reports, etc.
- Storage via Supabase Storage or an external bucket
- File versioning for revised submissions
- Document previewing within the browser

This would require expanding the schema to include a *Documents* table with metadata such as file size, type, and upload date.

- (2) **Expanded Role-based Access Controls:**

While the current RLS policies enforce essential user and admin separation, further improvements could include:

- Multiple administrator tiers (super admin, admin, reviewer, read-only auditor, etc).
- Institutional-level access (team members managing the same grant) through institution verification.
- Audit logs for compliance tracking.

This would help GrantGenie be in compliance with real-world requirements and improve tracing / auditing capabilities.

- (3) **Performance Scalability and Caching:**

As the number of users and grants scale up, the following performance-enhancing concepts could be implemented:

- Revisit database indexing to match usage patterns.

- Frequently accessed queries could be cached server-side using React Server Components and Suspense.
- Implement test queries and analyze / optimize slow ones using Supabase's Query Advisor.
- Batch or pool connections to improve Supabase Edge Functions performance.

Load testing, caching, and other performance improvements would significantly increase GrantGenie's viability as a real-world system.

#### (4) **Automated Testing and CI/CD Integration:**

Although the first release's development process relied primarily on manual testing, future work should include:

- Automated unit tests for front-end, back-end, and Supabase/database.
- Integration test for full-stack, end-to-end flow, using Selenium. Should be triggered on pull requests.
- A GitHub Actions or GitLab CI pipeline.
- Automated deploy previews, to ensure version release stability.

This would greatly improve maintainability, drastically increase developing output, and prevent regression bugs as the system grows.

#### (5) **Building standalone infrastructure:**

Currently, the frontend server is hosted locally by the user, and backend features are handled by Supabase and Google. In the future, an enterprise app might want to build its own infrastructure:

- The training of a new LLM model would allow for further specialization of the model for grant management tasks. However, Google Gemini works reasonably well for the required tasks, and training a unique LLM is beyond the scope of this project.
- The hosting of the LLM on a private server would allow greater data privacy. It is unwise to enter sensitive financial data into a publicly accessible LLM. Though current features do not require it, data redaction and additional safeguards would need to be implemented to ensure data such as bank accounts and Social Security Numbers are not accidentally fed into Google Gemini. However, since the data is currently used for testing, this is not a current issue anyway.
- The hosting of backend infrastructure on a private server would allow
  - The hosting of GrantGenie's frontend on a webserver would allow users to access GrantGenie without downloading or setting up their own server. Though not necessary for a research project, it would be imperative for a public launch.

Currently, the resources to build, maintain, and secure a GrantGenie server for public use is unavailable. The security and computing resources of Google Gemini and Supabase are by far sufficient for private and small-scale public use of GrantGenie at this current time, but a transition to privately hosted infrastructure would be needed for an enterprise-level app.

## 15 Installation

The following includes instructions on the installation and use of GrantGenie.

### 15.1 Installation

- (1) Visit <https://github.com/joshnorlin/GrantGenie>.
- (2) Download GrantGenie as a .zip file and extract the folder.
- (3) Open a terminal and navigate to GrantGenie/Frontend/
- (4) Run "npm install"
- (5) Run "npm run dev" to start the server
- (6) Type "o + Enter" to start open in a browser or visit <http://localhost:5173/>

### 15.2 Getting Started

To get started, follow the instructions on the landing page to create an account. This will require an email address. Once logged in, explore GrantGenie's features using the tabs on the top bar. Although GrantGenie is designed to be intuitive, more information and documentation on installation and use can be found throughout this report and in "GrantGenie/Frontend/Readme.md" within the source code files if issues arise.

## 16 Conclusion

GrantGenie seeks to allow PIs to create and manage grants while complying with federal and institutional rules. This project will ensure platform-integrity, security, and grant compliance through an integrated front-end and hosted back-end / database architecture.

Users can create grants, enter and verify transactions, and manage budgets. Users will also be able to export financial information to use for auditing and compliance purposes. Ultimately, GrantGenie provides a simple, secure and reliable grant management experience.

## 17 Source Code

GrantGenie's source code can be found at <https://github.com/joshnorlin/GrantGenie>.